CHAPTER 0

The Foundation

Building the kitchen before you cook anything

What this chapter is about

Before we scan a single device, we need a place to *store* what we find, a way to *send orders* to the thing doing the scanning, a *safe vault* for customer passwords, and a *ledger* that records who did what. This chapter builds that plumbing. Nothing in this chapter actually touches a network — it's all setup.

> ***Real-life analogy.*** *Imagine you're starting a courier company. Before you accept a single parcel, you need: warehouses where parcels are stored, a dispatch radio to tell drivers where to go, ID cards for each driver, locked envelopes for sensitive contents, and a paper logbook so you can prove later "yes, parcel #482 went to Mr. Sharma at 3:14 pm." If you start delivering before any of that exists, you'll lose parcels, mix up customers, and have no answer when something goes wrong. Chapter 0 is "open the depot before the trucks roll."*

Words you'll see (and what they mean)

> **API (Application Programming Interface).** A doorway one program uses to talk to another. Our NestJS backend exposes an API; the Go agent calls it.

> **NestJS.** The web framework on our server (TypeScript). It's already in your repo at `backend/BE`.

> **Postgres (PostgreSQL).** The database we already use. Stores everything permanently.

> **RLS (Row-Level Security).** A built-in Postgres feature that says "every row in this table belongs to one tenant; if a query doesn't say the right tenant ID, it sees zero rows." Like

putting a name tag on every file in a shared filing cabinet so only the right customer can pull theirs.

Tenant. One customer organisation. Acme Corp is a tenant; Globex is another tenant. They share our software but never see each other's data.

Collector. The Go program that runs *inside* a customer's network and does the actual scanning. We are extending the existing agent to also work as a collector.

WebSocket. An always-open phone line between collector and server (instead of "hang up and redial" HTTP). You already use `gorilla/websocket` in the agent.

CIDR (Classless Inter-Domain Routing). A way to write a range of IP addresses. `192.168.1.0/24` means "all 256 addresses from 192.168.1.0 to 192.168.1.255." We use CIDRs to say "tenant Acme is allowed to scan these ranges and no others."

Envelope encryption. Encrypting a key with another key. We encrypt the customer's firewall password with a per-tenant key (DEK = Data Encryption Key), and we encrypt that DEK with a master key (KEK = Key Encryption Key) held in a vault. To decrypt the password you need both. If the database is leaked, the passwords are useless without the master key.

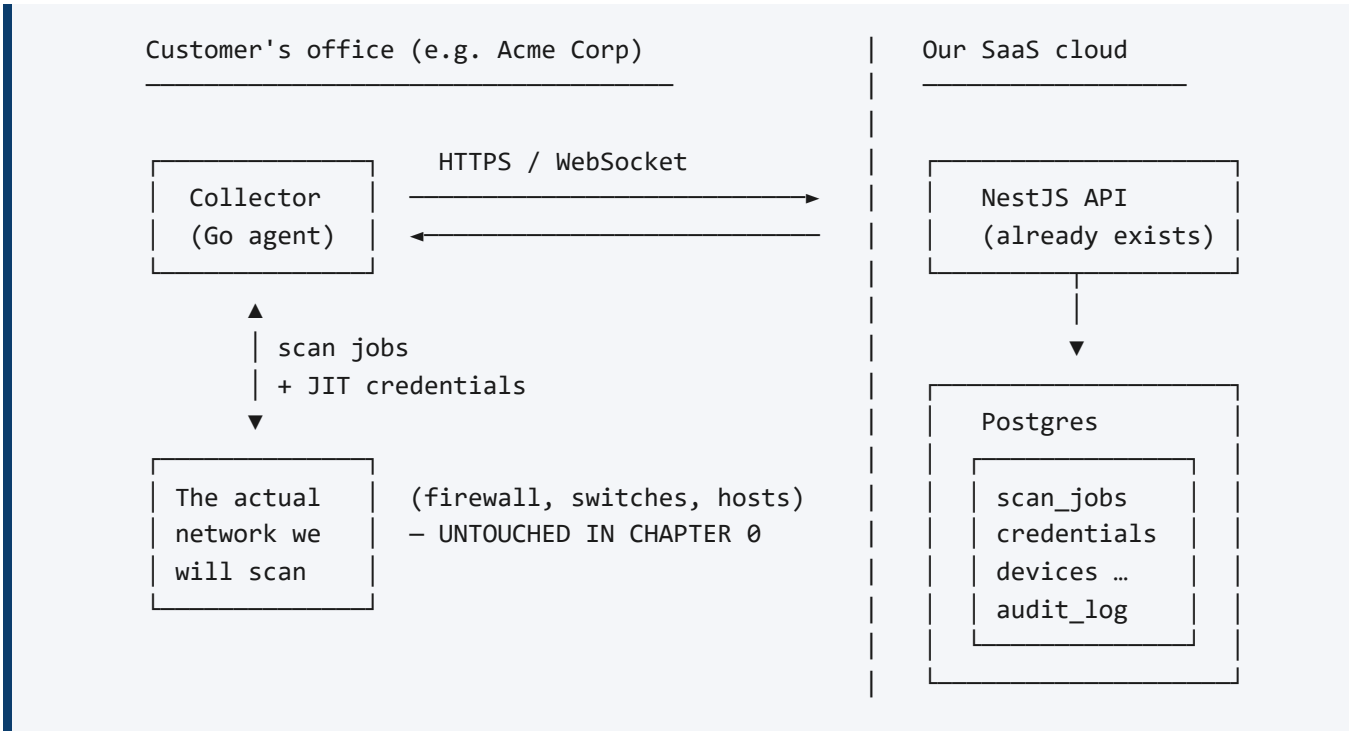
JIT (Just-In-Time) delivery. The collector receives a password only at the moment it needs it, holds it in RAM, and zeros it out the second the job ends. The collector never writes it to disk.

Audit log. A write-only diary. "On 2026-05-07 at 14:23, user shubham@humancloud.co.in triggered a firewall scan on Acme's site Mumbai-HQ." You can read it forever; you can never edit it.

LISTEN/NOTIFY. Postgres's built-in messaging — when one process tells the database "NOTIFY new_job", every other process that ran "LISTEN new_job" instantly gets a ping. Free real-time messaging without adding Redis or Kafka.

SKIP LOCKED. A Postgres trick to let many workers grab jobs from the same table without stepping on each other. Worker A locks job 1, Worker B's query says "give me a job, but skip any that are locked" — so Worker B picks up job 2 instantly, no fighting.

What we're building (the picture)



In Chapter 0 we set up *only* the right-hand side and the message channel. The left-hand side is wired but does nothing useful yet — it just answers "yes I'm here" and echoes test jobs back.

Step-by-step plan

Step 1 — Create the database tables

Add a new NestJS module called `discovery` at `backend/BE/src/discovery/`. Inside it, create these TypeORM entities (one TypeScript class per table):

Table	What it stores (in plain English)
`site`	One physical office or branch belonging to a tenant. "Acme HQ Mumbai", "Acme Branch Pune".
`subnet`	An address range used at a site. "192.168.10.0/24 is Acme HQ floor 3."
`device`	One discovered thing — a router, switch, server, printer, laptop.
`interface`	One network port on a device — like "eth0 on web-prod-01".
`ip_address`	An IP that's bound to an interface, with timestamp.

`mac_address`	The hardware address of an interface.
`neighbor_edge`	"Switch-A port Gi1/0/3 is cabled to Switch-B port Gi2/0/14." This is what makes the topology a graph.
`route_entry`	One row from a router's routing table.
`open_port`	"On 10.0.5.42, TCP port 22 is open and the banner says OpenSSH_8.4."
`discovery_session`	One scan run. "Crawler started at 14:00, finished 14:18, found 312 devices."
`observation`	One single fact, with provenance. "I saw IP 10.0.5.42 on MAC aa:bb:.. at 14:03 via collector-7." This is the secret sauce — every other table is a view over observations.
`credential`	An encrypted password/community/SSH key blob.
`scan_job`	An order: "go run firewall ingest on Acme's HQ firewall."
`audit_log`	Append-only diary of who triggered what.

Every table gets a `tenant_id UUID NOT NULL` column. Every table gets `created_at` and `updated_at`. `observation` additionally gets `seen_at`, `source_collector_id`, `session_id`.

Step 2 — Turn on Row-Level Security

For every discovery table, run this SQL once:

```
ALTER TABLE device ENABLE ROW LEVEL SECURITY;
CREATE POLICY device_tenant_isolation ON device
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Then in the NestJS connection-pool wrapper, before every request handler runs, do:

```
await queryRunner.query(`SET app.tenant_id = $1`, [req.tenantId]);
```

Now even if a developer forgets to add `WHERE tenant_id = ?` in a query, Postgres itself refuses to return another tenant's rows. Two locks on the same door.

Step 3 — Build the credential vault

Create a service `CredentialVaultService`:

- **Encrypt:** caller passes plaintext + tenant_id. Service generates a fresh DEK if the tenant doesn't have one yet, encrypts the secret with the DEK using AES-256-GCM, stores the ciphertext.
- **Decrypt:** caller passes ciphertext + tenant_id. Service unwraps the DEK, decrypts, returns plaintext, writes one row to `audit_log`.
- **Master key:** for development read it from an env var `ITOM_MASTER_KEY` (32 random bytes, base64). For production, swap to AWS KMS or HashiCorp Vault behind the same interface — same code, different driver.

Step 4 — Build the scan-job dispatcher

The flow we want:

```
User clicks "Run firewall ingest"
      │
      ▼
POST /v1/discovery/scans  (NestJS)
      │
      ├──► INSERT INTO scan_jobs (...) RETURNING id;
      │
      ├──► NOTIFY collector_<collectorId>, '{ "jobId": "..." }';
      │
      ▼
Collector (already LISTENing) wakes up
      │
      ├──► SELECT ... FROM scan_jobs WHERE id = ...
      │       FOR UPDATE SKIP LOCKED;
      │
      ├──► do the work
      │
      └──► stream results back over WebSocket as ScanJobChunk frames
```

For Chapter 0 we don't *do* the work yet — the collector just answers `{"pillar":"noop","echo":true}` and we confirm a fake observation row appears in the database tagged to the right tenant.

Step 5 — Add WebSocket message types

In `backend/agent/internal/wsproto/wsproto.go` add four new frame kinds:

Frame	Direction	What it carries
`ScanJobAssign`	server → collector	Job ID, pillar, target spec, wrapped credentials, tenant CIDR allowlist (signed)

`ScanJobAck`	collector → server	"Got it, starting"
`ScanJobChunk`	collector → server	A batch of observations (NDJSON)
`ScanJobDone`	collector → server	"Finished, summary: 312 devices"
`ScanJobError`	collector → server	"Failed because ..."

Step 6 — Build the CIDR allowlist enforcer in the collector

Every IP or hostname the collector touches must be inside the tenant's authorized CIDRs. The check happens *twice*:

1. The server includes the tenant's allowlist in `ScanJobAssign`, signed with the server's private key.
2. The collector verifies the signature, then for every target it generates, asks "is this IP in any allowlisted CIDR?" If no → refuse, write a `ScanJobError`, and write an `audit_log` row.

This is "defence in depth" — even if the server has a bug that sends the wrong CIDR list, the collector won't accidentally scan Acme's competitor.

How we test Chapter 0

Test 1 — round trip a fake job.

1. Create two tenants: Acme and Globex.
2. Register one collector under Acme.
3. POST a scan job with `pillar=noop` and target `192.168.1.1`.
4. Confirm: collector receives `ScanJobAssign`, sends `ScanJobAck`, sends one fake `ScanJobChunk`, sends `ScanJobDone`. Database has an `observation` row tagged `tenant_id = Acme`.

Test 2 — RLS actually works.

1. Open a database session as the application user.
2. Without setting `app.tenant_id`, run `SELECT * FROM device;` → must return zero rows.

3. Set `app.tenant_id` to Acme's ID, repeat → must return only Acme's rows.

Test 3 — credential vault round-trip.

1. Encrypt the string `"hunter2"` for tenant Acme.
2. Inspect the database — the stored value must NOT contain "hunter2".
3. Decrypt → must return exactly `"hunter2"`.
4. Confirm one row in `audit_log`.

Test 4 — CIDR allowlist refuses out-of-scope.

1. Configure Acme's allowlist as `10.0.0.0/8` only.
2. Send a job to scan `8.8.8.8`.
3. Collector must refuse with `ScanJobError`; database must have a refusal row in `audit_log`; `8.8.8.8` must never receive a packet.

Things that can go wrong (and what to watch for)

Forgetting to SET app.tenant_id. If your connection-pool wrapper doesn't run that command on every checkout, RLS silently returns zero rows for everything and you'll think the database is broken. Add a startup test that fails loudly if the variable isn't set.

Storing the master key in the same database as the encrypted blobs. Defeats the entire point. Master key lives in env / KMS / Vault. Never in Postgres.

NOTIFY payload is bigger than 8KB. Postgres truncates it. Send only the job ID in the NOTIFY; collector fetches the rest with `SELECT`.

Collector trusts the server's CIDR list blindly. If you skip the signature, a compromised server (or a misconfiguration) can point your scanner at any address. Always sign + verify.

Reusing the existing agent's WebSocket without distinguishing "agent mode" from "collector mode". Make the role explicit in the registration message; don't let "host metrics" jobs and "discovery" jobs share the same code path silently.

Exit checklist (you can move to Chapter 1 only when ALL are true)

- ☐ Two tenants exist, each with a distinct UUID.
- ☐ A collector registered to a tenant talks to the server over WebSocket and stays connected for ≥ 10 minutes without dropping.
- ☐ A `noop` scan job round-trips and produces an observation row tagged with the right tenant.
- ☐ With `app.tenant_id` unset, every discovery query returns zero rows.
- ☐ A credential encrypts and decrypts correctly; ciphertext does not contain plaintext substring.
- ☐ A scan job targeting an out-of-allowlist IP is refused, audited, and never sends a packet.
- ☐ Killing the collector mid-job leaves the job `status=running`, and a stale-job sweeper marks it `timeout` after a configured TTL.

Recap card — Chapter 0 in 5 sentences

1. Build the database tables for sites, subnets, devices, interfaces, MACs, IPs, edges, ports, observations, sessions, credentials, jobs, and audits.
2. Turn on Postgres Row-Level Security so a missing tenant filter can never leak data.
3. Build a credential vault that encrypts secrets with envelope encryption and writes an audit row on every decrypt.
4. Wire scan jobs through Postgres LISTEN/NOTIFY + SKIP LOCKED + your existing WebSocket — no Redis needed.
5. Make the collector verify a signed CIDR allowlist and refuse anything outside it; test that the round-trip works with a `noop` job before you let any pillar do real work.